

ALGORITHM ANALYSIS

Unit 02

Presented By: Saroj Bhandari

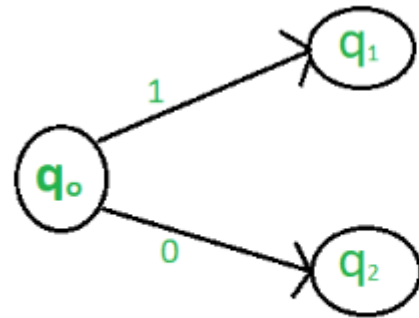
Syllabus

- Deterministic and Non-deterministic Algorithm
- Divide and Conquer Algorithm
- Series and Parallel Algorithm
- Heuristic and Approximate Algorithms

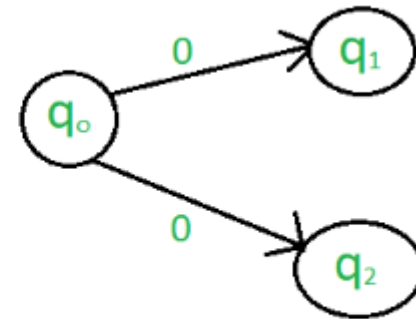
Difference between Deterministic and Non-deterministic Algorithms

- In a **deterministic algorithm**, for a given particular input, the computer will always produce the same output going through the same states but in the case of the **non-deterministic algorithm**, for the same input, the compiler may produce different output in different runs.
- The non-deterministic algorithms can show different behaviors for the same input on different execution and there is a degree of randomness to it.
- Examples of deterministic algorithms are Merge Sort, Binary Search and most of the other algorithms that we generally study and that do not include randomness.
- Example of non-deterministic algorithm is Randomized Quick Sort.

Example



Deterministic Algorithm



Non-Deterministic Algorithm

Differences:

Deterministic Algorithm	Non-deterministic Algorithm
A deterministic algorithm is one whose behavior is completely determined by its inputs and the sequence of its instructions.	A non-deterministic algorithm is one in which the outcome cannot be predicted with certainty, even if the inputs are known.
For a particular input, the computer will give always the same output.	For a particular input the computer may give different outputs on different execution.
Can determine the next step of execution.	Cannot determine the next step of execution due to more than one path the algorithm can take.
Operation are uniquely defined.	Operation are not uniquely defined.
Like linear search and binary search	like travelling salesman person and evaluate boolean parenthesis to true.
Deterministic algorithms usually have a well-defined worst-case time complexity.	Time complexity of non-deterministic algorithms is often described in terms of expected running time.
Deterministic algorithms are entirely predictable and always produce the same output for the same input.	Non-deterministic algorithms may produce different outputs for the same input due to random events or other factors.
Deterministic algorithms usually provide precise solutions to problems.	non-deterministic algorithms often provide approximate solutions to the problems.
Examples of deterministic algorithms include sorting algorithms like bubble sort, insertion sort, and selection sort, as well as many numerical algorithms.	Examples of non-deterministic algorithms include probabilistic algorithms like Monte Carlo methods, genetic algorithms, and simulated annealing.

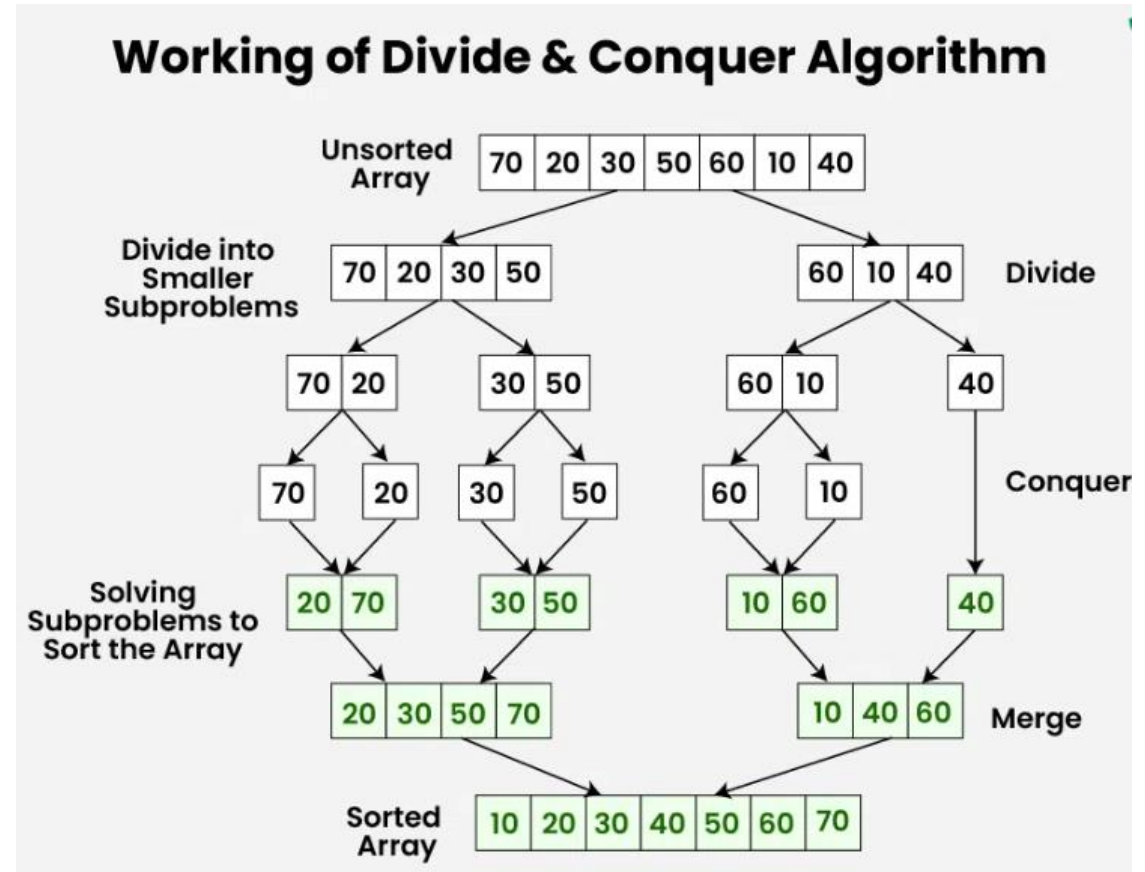
Divide and Conquer Algorithm

- **Divide and Conquer Algorithm** is a problem-solving technique used to solve problems by dividing the main problem into subproblems, solving them individually and then merging them to find solution to the original problem.
- Divide and Conquer is mainly useful when we divide a problem into independent subproblems.
- If we have overlapping subproblems, then we use Dynamic Programming.

•

Working of Divide and Conquer Algorithm

- Divide and Conquer Algorithm can be divided into three steps:
- Divide
- Conquer
- Merge.



Explanation

1. Divide:

- Break down the original problem into smaller subproblems.
- Each subproblem should represent a part of the overall problem.
- The goal is to divide the problem until no further division is possible.
- In Merge Sort, we divide the input array in two halves. Please note that the divide step of Merge Sort is simple, but in Quick Sort, the divide step is critical. In Quick Sort, we partition the array around a pivot.

2. Conquer:

- Solve each of the smaller subproblems individually.
- If a subproblem is small enough (often referred to as the “base case”), we solve it directly without further recursion.
- The goal is to find solutions for these subproblems independently.
- In Merge Sort, the conquer step is to sort the two halves individually.

3. Merge:

- Combine the sub-problems to get the final solution of the whole problem.
- Once the smaller subproblems are solved, we recursively combine their solutions to get the solution of larger problem.
- The goal is to formulate a solution for the original problem by merging the results from the subproblems.

Examples of Divide and Conquer Algorithm

1. Merge Sort:

- We can use Divide and Conquer Algorithm to sort the array in ascending or descending order by dividing the array into smaller subarrays, sorting the smaller subarrays and then merging the sorted arrays to sort the original array.

2. Quicksort:

- It is a sorting algorithm that picks a pivot element and rearranges the array elements so that all elements smaller than the picked pivot element move to the left side of the pivot, and all greater elements move to the right side. Finally, the algorithm recursively sorts the subarrays on the left and right of the pivot element.

Applications of Divide and Conquer Algorithm

- Binary Search is an efficient algorithm for finding an element in a sorted array by repeatedly dividing the search interval in half. It works by comparing the target value with the middle element and narrowing the search to either the left or right half, depending on the comparison.
- Quicksort is a sorting algorithm that picks a pivot element and rearranges the array elements so that all elements smaller than the picked pivot element move to the left side of the pivot, and all greater elements move to the right side. Finally, the algorithm recursively sorts the subarrays on the left and right of the pivot element.
- Merge Sort is also a sorting algorithm. The algorithm divides the array into two halves, recursively sorts them, and finally merges the two sorted halves.
- Closest Pair of Points The problem is to find the closest pair of points in a set of points in the x-y plane. The problem can be solved in $O(n^2)$ time by calculating the distances of every pair of points and comparing the distances to find the minimum. The Divide and Conquer algorithm solves the problem in $O(N \log N)$ time.

1. Series Algorithm (Sequential Algorithm)

- ❑ A **Series Algorithm** is an algorithm in which **instructions are executed one after another** in a specific order. Each operation is performed only **after the previous one is completed**. There is no overlap or concurrency in the execution.
- ❑ **Example:** Reading a book page by page.

2. Parallel Algorithm

- ❑ A **Parallel Algorithm** is an algorithm where **multiple operations are carried out simultaneously** using multiple processors or threads. It divides a large problem into **smaller independent tasks** and processes them **concurrently**, thus saving time.
- ❑ **Example:** A group of people reading different chapters of the same book at the same time.

Difference between Series and Parallel Algorithm

Feature	Series Algorithm	Parallel Algorithm
Execution	One instruction at a time	Multiple instructions at the same time
Speed	Slower (especially for large problems)	Faster (utilizes multiple processors/cores)
Resource Usage	Uses a single CPU or core	Uses multiple CPUs or cores
Design Complexity	Simple to design and debug	More complex to design and debug
Dependency	Each step may depend on the previous one	Tasks can be independent and run simultaneously
Example	Traditional loops (e.g., summing in a loop)	Matrix multiplication using threads
Best For	Simple or small tasks	Large data, high-performance computing

Heuristic Algorithm

- A **Heuristic Algorithm** is a method that finds a good enough solution quickly by using practical techniques or rules of thumb. It **does not guarantee** the best or optimal solution, but it **aims for efficiency** in solving complex problems where exact methods are too slow or impractical.
- **Examples:**
 - Greedy algorithms
 - Nearest neighbor algorithm in the Traveling Salesman Problem (TSP)
- **Used in:**
 - AI and robotics
 - Game playing (like chess)
 - Route planning (GPS)

2. **Approximate** Algorithm

- An **Approximate Algorithm** is a type of algorithm that **finds near-optimal solutions to optimization problems** where finding an exact solution is too costly (NP-Hard problems). These algorithms come with a **guarantee or bound** on how far the result is from the optimal.
- **Examples:**
 - Approximation for TSP within 1.5 times the optimal (using Minimum Spanning Tree)
 - Vertex cover approximation algorithms
- **Used in:**
 - Network design
 - Scheduling
 - Resource allocation

Difference between Heuristic and Approximate Algorithms

Feature	Heuristic Algorithm	Approximate Algorithm
Goal	Find a good solution quickly	Find a solution close to the optimal
Optimal Guarantee	No guarantee of optimality	Has a known bound from the optimal solution
Speed	Very fast, even for large problems	Fast, but may be slower than heuristics
Based On	Practical rules or intuition	Mathematical approximation and analysis
Result Quality	Often good but unpredictable	Always within a known range of the optimal result
Use Case	Games, AI, real-time systems	NP-Hard problems like TSP, Knapsack
Example	Nearest Neighbor in TSP	MST-based TSP Approximation

Thank You !!!!!